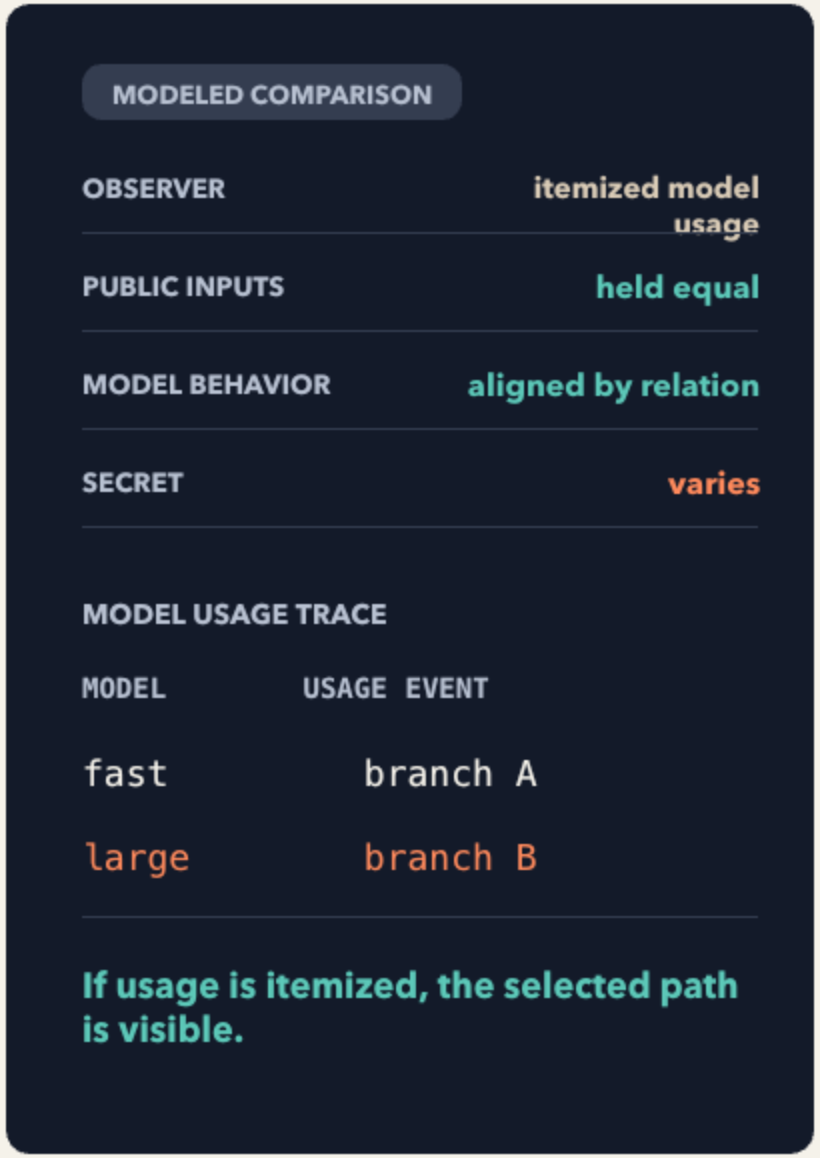
OrchLang

Replayable Contracts for Token Usage and Request-Equivalent LLM Workflows

Nambi Rajan M · 24BAI0072 · Vellore Institute of Technology



Equal means can hide different usage laws

Both branches average two output tokens. The finite usage distributions are disjoint.

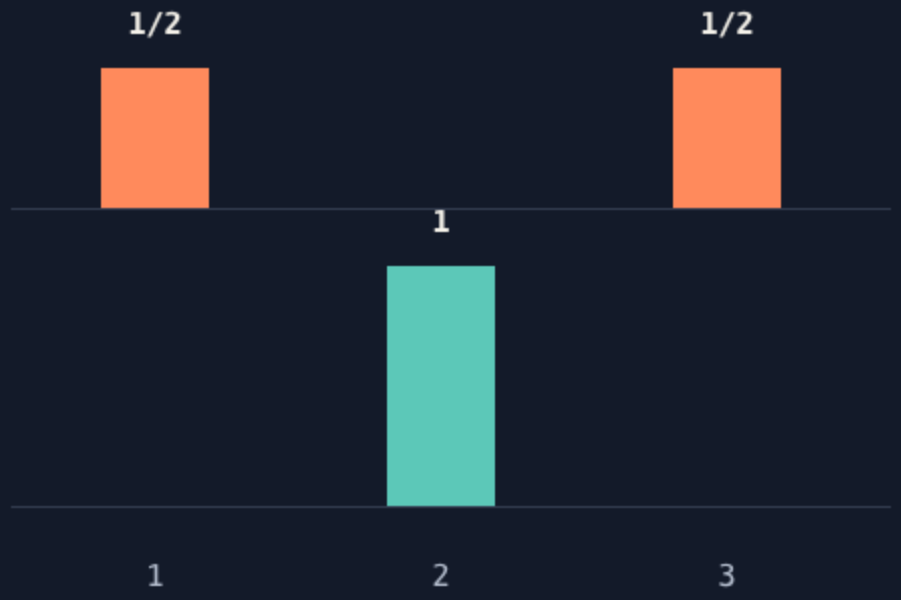
OUTPUT USAGE PER CALL · tokens

TEMPLATE A

P(1)=1/2 P(3)=1/2

TEMPLATE B

P(2)=1



EXPECTED OUTPUT

2 = 2

TV = 1

I(secret ; usage) ≈ 1 bit

Uniform prior · finite authored kernel · H=1

SOURCE examples/channels/same_mean_distinct_laws.graph.json

Aggregate totals can hide call order

At horizon two, the aggregate law matches while the ordered trace differs.

TWO CALLS · SAME MODEL · BOTH RUNS STOP

SECRET = 0

`output usage: 1 → 3`

SECRET = 1

`output usage: 3 → 1`

ORDERED TRACE

`(1,3) ≠ (3,1)`

`TV = 1`

SECRET = 0

`calls=2 · input=2 · output=4`

SECRET = 1

`calls=2 · input=2 · output=4`

AGGREGATE VECTOR

same billing row

`TV = 0`

Exact channel result at horizon H=2 under the supplied finite kernel

SOURCE examples/channels/aggregate_hides_order.graph.json

Three questions: budget, trace and channel

Each analysis fixes a different relation and reports a different result.

01

Unary bound

How much can one run use?

`C(I) ≤ declared envelope`

02

Paired trace

Does changing a secret change usage?

`trace(run₀) = trace(run₁)`

03

Finite channel

What does an observation reveal?

`TV, vulnerability, MI`

Different inputs, different assumptions, different outputs

SOURCE docs/PAPER.md §§3–5 · docs/FORMAL_CONTRACT.md · docs/PROBABILISTIC_CHANNELS.md

Four input paths keep claims separate

Compiler, graph, source and finite-kernel analyses use distinct contracts.

INPUT ARTIFACT	ANALYSIS	CLAIM SCOPE	
C++ COMPILER **bounded OrchLang AST**		flow + unary cost + request relation	finite syntax
JSON GRAPH **versioned finite-control model**		product closure + certificate replay	productive call cycles
SOURCE PATHS **OrchLang subset / Python AST**		export, replay, offline LangGraph	authored restricted subsets
FINITE KERNELS **graph + rational kernel**		DP channels + bisimulation + exact row span	explicit finite stores

These paths share assumptions and report formats; the source and proof boundaries remain separate.

SOURCE docs/PAPER.md §§2–5

Equal caps still permit different realized usage

A secret can choose between public arguments with different concrete lengths.

SECRET = 0

```
fixed public value: left_source
cap = 32 tokens
realized length = n
input usage = 3 + n
shared-oracle output = o₀
```

SECRET = 1

```
fixed public value: right_source
cap = 32 tokens
realized length = m
input usage = 3 + m
shared-oracle output = o₀
```

same per-call envelope: input ≤ 35 · output ≤ 4 · calls = 1

SOURCE examples/research/rejected/equal_bounds_different_inputs.orch

The bounded compiler compares resolved requests

A sufficient local check preserves identity across aliases, values and dependent calls.

SECRET ALTERNATIVE 0

```
M = (provider, actual ID, cap, price)
Q = exact template + ordered atoms
```


SECRET ALTERNATIVE 1

```
same resolved M
same Q and result position
```

15 / 15

balanced source-derived projections admitted

8 / 15

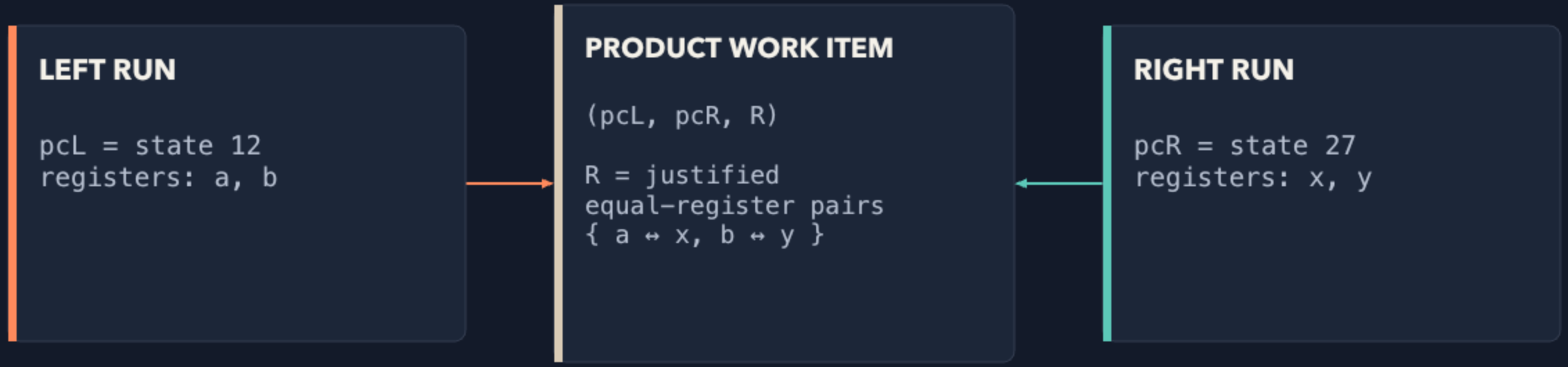
preceding frozen version

45 authored variants · 30 source-bound reports checked · partial projections

SOURCE src/relational.cpp

The cyclic checker relates two control states

A product node records control on both sides and justified equal values.



The relation is updated only when a matched call and shared Ω justify equal results.

SOURCE audit/orchgraph.py

Secret choices branch universally in the product

Public predicates synchronize on related values; every secret successor pair must pass.

SECRET GUARD

Explore every left/right successor pair.

PUBLIC GUARD

Same normalized test on R-related values; take the same arm.

CALL / CALL

Match full contract, template, ordered atoms; update R after Ω.

STOP / STOP

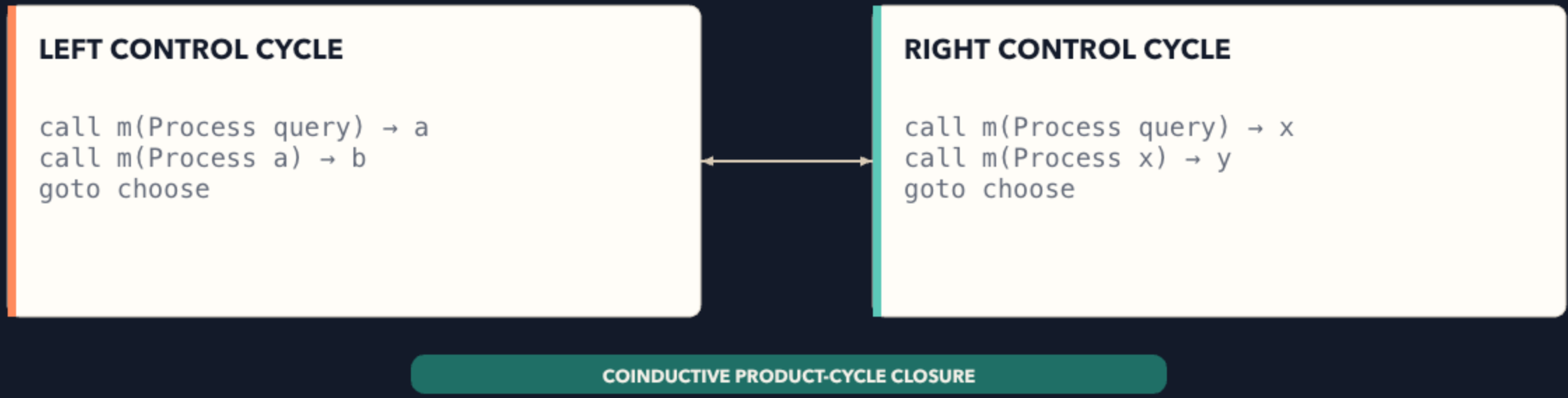
Both stop together. A stop/call mismatch rejects.

Reachable product worklist · universal secret choices · fail closed on unsupported states or state-cap exhaustion

SOURCE audit/orchgraph.py

Productive cycles establish usage-prefix equality

A reachable product state can close after its register relation stabilizes.



Closes all finite usage prefixes under Ω. It does not prove termination or a finite total-token / price bound.

24,000 calls in the finite probe · 200 runs hit the 120-call cap

SOURCE examples/graphs/accepted/unbounded_productive_cycle.json

Overwriting a result erases its old pairing

The third request cannot use a relation that an overwrite invalidated.

LEFT

CALL 0

```
#0 query → a
```

CALL 1

```
#1 query → a  (overwrite)
```

CALL 2

```
#2 request(a) → b
```

RIGHT

CALL 0

```
#0 query → x
```

CALL 1

```
#1 query → y
```

CALL 2

```
#2 request(x) → y
```

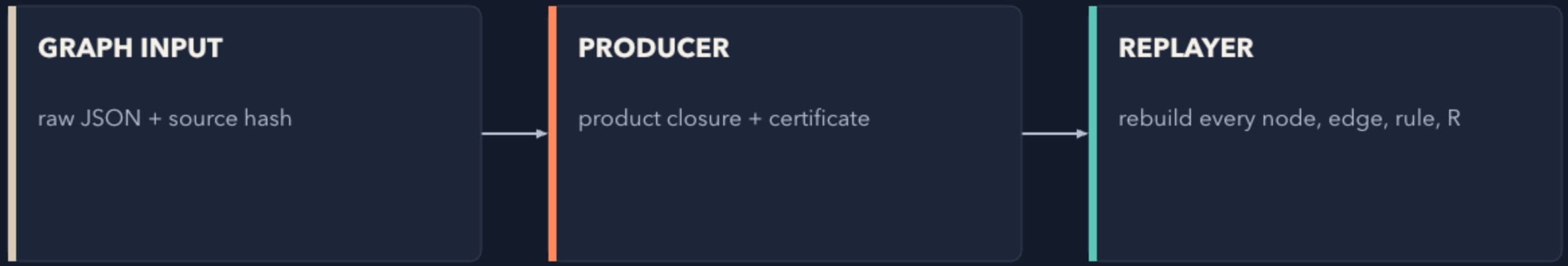
After overwrite: a ↔ y, not x. The third arguments are not proved equal.

```
One finite mock witness at event 2: input usage 10 vs 15 · not a provider invoice.
```

SOURCE examples/graphs/rejected/overwritten_result_relation.json

A separate replayer rebuilds the graph closure

Raw source binding and every required product successor are checked again.



VERIFIED · 6/6 accepting certificates replayed

Replay checks certificate closure under the graph rules; it does not verify graph-to-source fidelity or mechanize the relational theorem.

SOURCE audit/replay_graph_certificate.py

The source bridge admits a small OrchLang subset

17 of 20 selected source files exported; retry syntax accounts for the exclusions.

20/20

source candidates

C++ gate passed

17/20

exported

3 retry cases outside graph v1

720

source/graph run pairs

complete modeled traces

0

trace differences

360 secret-trace comparisons

An experimental source adapter with a narrow subset; hashes bind files but do not verify translation.

SOURCE docs/SOURCE_GRAPH_BRIDGE.md

Source-level StateGraph examples reach LangGraph

A restricted Python AST frontend was compared with original-source executions.

20

frontend tests

static AST subset

36

original-source runs

authored StateGraph functions

30 + 6

matched comparisons

accepted traces + rejection probes

0

comparison differences

across the reported source grid

Restricted Python AST programs only; no general source-to-graph correctness theorem.

SOURCE integrations/langgraph/source_frontend/README.md

The pinned backend found no trace differences on its grid

The finite execution grid includes both stopped and horizon-truncated runs.

1.2.14

pinned LangGraph

offline runtime

888

executions

864 stopped · 24 truncated

4,528

call-event pairs

request and usage traces

0

trace differences

on the finite test grid

Pinned local backend with a deterministic offline oracle; no provider call or billing record.

SOURCE integrations/langgraph/requirements.lock

Full request text was checked in bounded compiler stress

A separate Python evaluator substitutes complete rendered requests under a controlled oracle.

264

generated C++ cases

144 accepted · 120 mutations rejected

24,192

independent reference runs

full rendered request text

0 / 13,824

mismatch / accepted pairs

request and usage traces

66,240

per-call accounting checks

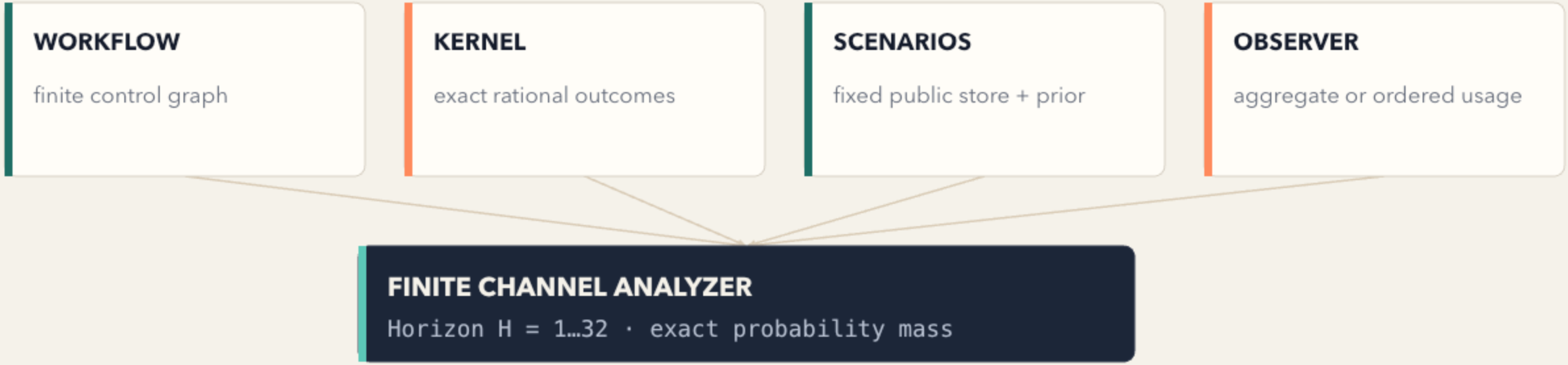
117 mutation witnesses · 3 unknown

C++ checker only compiles cases; a separate Python interpreter substitutes full text. Synthetic cases, not a proof or provider test.

SOURCE docs/RELATIONAL_STRESS.md

The channel analyzer takes finite kernels as input

The tool evaluates explicit probabilities for fixed public stores and selected secrets.

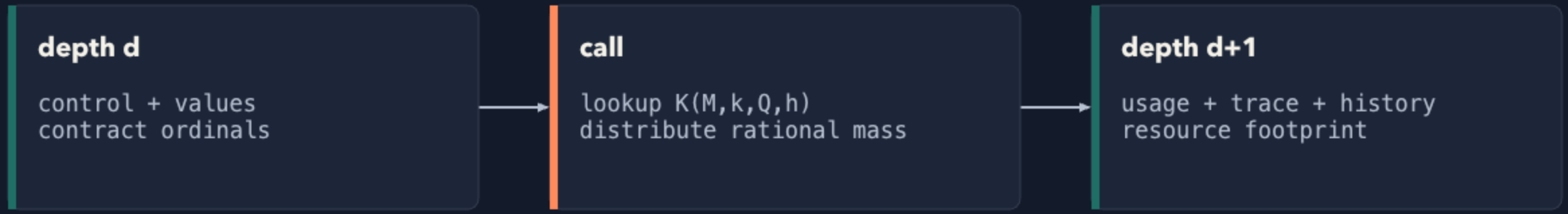


Probabilities describe the supplied finite kernel, not a learned or provider-certified distribution.

SOURCE audit/probabilistic_channels.py

Dynamic programming keeps exact probability mass

State merging is safe only when all future-relevant and observed fields agree.



MERGE CONDITION

Every future-relevant value and observed/resource field is equal

REFERENCE

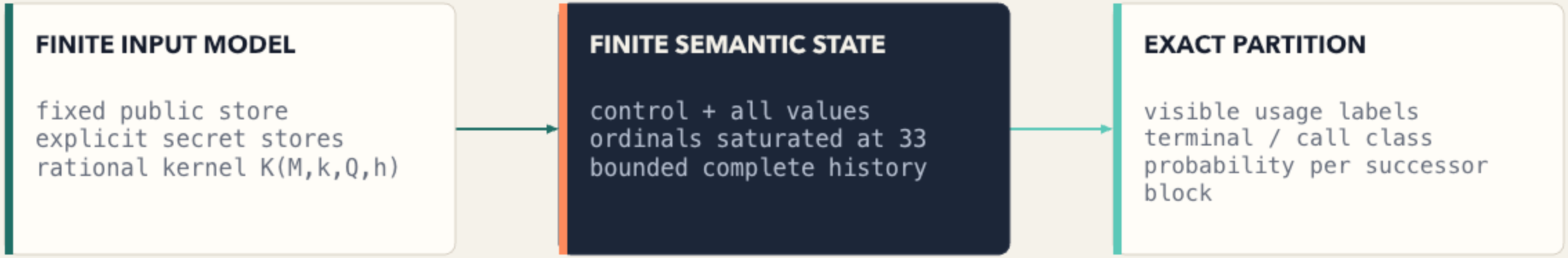
No-merge DFS path enumeration

Rational caps or state/path limits return unknown

SOURCE audit/probabilistic_channels.py

Bisimulation is sufficient, not complete

A shared stable block proves equality; a split leaves the result open.



SAME INITIAL BLOCK

sufficient for equal ordered usage laws at every finite call horizon

SPLIT BLOCKS

inconclusive; not a trace-law witness

Success-only semantics · finite explicit stores · no cumulative cost or infinite-run claim

SOURCE audit/stochastic_bisimulation.py · docs/STOCHASTIC_BISIMULATION.md · Larsen & Skou 1991

A separate replayer checks the stable partition

Finite-state certificates bind the graph, kernel, scenarios and partition obligations.

5 / 6

proved / inconclusive

11 authored finite fixtures

11/11

normal certificates replayed

independent stable-partition check

60

exact DP cross-checks

H=1, 2, 4, 8, 16, 32 · two observers

120

scenario histograms

all unit mass · all comparisons matched

17 producer tests · 12 replayer tests · 11/11 fresh cap probes UNKNOWN · split blocks are inconclusive.

SOURCE audit/stochastic_bisimulation.py

Exact trace equivalence resolves a split

One fixture has ordered TV 0 at H=2; exact span closure proves all finite horizons.

SECRET BRANCH A · RESPONSE-DEPENDENT

FIRST returns output count 1. Red or blue (½ each) becomes the NEXT request.
Red yields 1; blue yields 3 output tokens.

SECRET BRANCH B · LITERAL REQUEST

FIRST also returns output count 1. NEXT("pink") returns 1 or 3 output tokens, each with probability ½.

STRONG BISIMULATION · INCONCLUSIVE

successor states split

ORDERED OUTPUT-COUNT WORDS · H=2 · TV=0

`(1,1): ½` `(1,3): ½`

EXACT RATIONAL ROW-SPAN · PROVED_EQUAL

$$\mu = eA - eB \qquad R = \mathrm{span}\{\ \mu\, A_w : w \text{ is a finite usage word}\ \}$$

Every basis row matches stopped and still-live mass. That proves every finite word. A nonzero residual gives a finite witness; a cap returns UNKNOWN.

Output-count pairs shown; contract and input usage also match · finite kernel only · no provider or total-cost claim

SOURCE audit/trace_equivalence.py · audit/replay_trace_equivalence.py · Tzeng 1992

Prior work narrows the contribution

Weighted-automata equivalence also predates this application.

FOUNDATION / SYSTEM	ESTABLISHED SCOPE	PROJECT BOUNDARY
Resource-aware IFC	Resource observations can enter noninterference guarantees.	A request-linked usage-event check.
RelCost / product programs	Relational cost reasoning and self-composition.	Resolved contracts and result-register pairing.
Expected resources / PPDL	Probabilistic costs and multi-call LLM/tool flows.	Usage laws under an explicit finite kernel.
QIF / trace probability	Information-theoretic leakage and finite trace laws.	A usage-only observer for model-call flows.
Weighted automata / APEX	Established finite-word equivalence algorithms and tools.	A specialized alphabet and replayable certificate.
AgentFlow / Agentproof	Static checks for structured agent workflows.	Different policies and observation properties.
Petri-net workflow cost	Author-reported unary cumulative LLM cost bounds.	Relational usage equivalence; adjacent unrefereed work.

Contribution framing: applied workflow software and evidence, not a new general theory.

SOURCE docs/PRIOR_ART_MATRIX.md · submission/research/current_review_sources.md

Relational claims depend on request and oracle contracts

Claims depend on full modeled requests, compatible accounting and aligned hidden state.

RELATIONAL CLAIM

Equal public-input identities
Exact modeled request match
Shared complete-outcome Ω
Productive cycles: finite prefixes

ACCOUNTING PREMISES

Input estimate uses byte divisor
Output obeys declared caps
Model token units align
Relevant history is included

EXTERNAL FACTS LEFT OPEN

Full provider serialization
Hidden account/cache/routing state
Real tokenizer and invoices
Termination and total cycle cost

The mathematical statement is only as broad as its input model and oracle contract.

SOURCE docs/FORMAL_CONTRACT.md

The artifact makes its limits inspectable

Reports, source hashes, replay tools, tests and documentation support independent review.

SOURCE	C++ compiler · graph checker · source adapters · finite channel analyzer
INDEPENDENT CHECKS	Source-cost auditor · graph, partition and trace-law replayers · no-merge DFS
EVIDENCE	Hash-bound reports · pinned runtime · offline fixtures · explicit unknowns
ACCESS	MIT license · citation metadata · install/build/reproduce instructions

Reproducible evidence supports review; it does not replace a soundness proof or venue decision.

SOURCE audit/final/manifest.json

A narrow usage contract supports reviewable claims

Static relations, exact trace laws and offline executions answer different questions.

STATIC	**Compare modeled usage traces across secret choices.**
CYCLIC	**Close productive graph cycles for every finite prefix.**
PROBABILISTIC	**Channels, bisimulation and row-span checks use finite kernels.**

Questions · then a local demonstration

SOURCE docs/PAPER.md

Formal usage observation and coupling

Event

(resolved contract, input usage, output usage).

Aggregate observer

Per contract: call count and input/output sums, plus stopped.

Ordered observer

Ordered events plus stopped or horizon-truncated status.

Shared oracle

Ω(M, k, Q, aligned relevant history) returns a complete outcome.

Cycle result

Productive cycles establish finite usage prefixes only.

Trace-law extension

Exact rational word equivalence compares STOP and live-prefix mass.

SOURCE docs/FORMAL_CONTRACT.md

Graph schema and certificate replay

Inputs

Typed bounded public and secret inputs; public identities seed R.

Models

Provider, actual model ID, output cap and optional price metadata.

Calls

Exact template; ordered literal/public/register atoms; result register.

Control

Finite states with call sequences and goto/public/secret/stop step.

Replay

Rebuild all reachable product nodes, rules, relations and edges.

SOURCE audit/orchgraph.py

Source support and exclusions

OrchLang bridge

Text inputs, mock models, text prompts/calls, supported guards.

Retry

Excluded because native retry/failure schedules are not graph v1.

Python subset

Static StateGraph builder; bounded state; synchronous call functions.

Runtime

Pinned LangGraph; deterministic offline response session.

Provenance

Hashes detect changed artifacts; they do not prove translation.

SOURCE docs/SOURCE_GRAPH_BRIDGE.md

Evidence ledger and denominators

Source projections

15 partial source patterns; same grid repeated across two mock modes.

C++ stress

264 cases; 13,824 accepted comparisons; 66,240 accounting checks.

Graph/LangGraph

Graph fixtures, source exports and pinned offline runtime use separate grids.

Finite channels

Finite kernels; exact DP/DFS histograms and channel metrics.

Bisimulation

Sufficient all-horizon partition; a split remains inconclusive.

Exact trace laws

12 cases · 6 equal / 6 unequal · 12 replayed

SOURCE bench/results/sourced.json

Likely examiner questions

Is this new general theory?	No. It applies established relational methods to a specific usage contract.
Do cycles imply a finite budget?	No. The result concerns finite trace prefixes under Ω.
Does the channel tool predict providers?	No. It analyzes explicitly supplied finite kernels.
Does LangGraph prove arbitrary source fidelity?	No. A restricted AST subset and authored grid are tested.
What does certificate replay verify?	Reconstructed finite semantics and exact span under versioned rules.
Why compare two observers?	Aggregation can erase call order that the trace observer retains.
What does a split partition mean?	Inconclusive. Exact trace equivalence can resolve some splits.
Is the row-span method new theory?	No. It applies established weighted-automata equivalence to this usage model.

SOURCE docs/FORMAL_CONTRACT.md

Selected foundations and adjacent systems

[1] Ngo et al. · resource-aware IFC · IEEE S&P 2017 · DOI 10.1109/SP.2017.53

[2] Çiçek et al. · Relational Cost Analysis · POPL 2017 · DOI 10.1145/3009837.3009858

[3] Ngo et al. · probabilistic resources · PLDI 2018 · Mandel et al. · PPDL · ICML 2026

[4] Smith · QIF foundations · FOSSACS 2009 · Basin & Köpf · adaptive side channels · CCS 2007

[5] Tzeng · probabilistic automata equivalence · SIAM J. Comput. 1992 · DOI 10.1137/0221017

[6] Kiefer et al. · probabilistic automata equivalence · CAV 2011 · APEX · CAV 2012

[7] Kiefer et al. · rational weighted automata · LMCS 2013 · Fijalkow et al. · trace laws · LMCS 2020

[8] Larsen & Skou · probabilistic bisimulation · I&C 1991 · Kamaleson et al. · finite-horizon refinement · SPIN 2016

[9] Weiss et al. · USENIX Security 2024 · Agentproof / AgentFlow · 2026 preprints

[10] Dickerson · LLM workflow Petri-net cost note · May 2026 · author-reported, unrefereed

Full citations, stable links and prior-art boundaries: submission/research/current_review_sources.md

SOURCE submission/research/current_review_sources.md · docs/PRIOR_ART_MATRIX.md

Explore finite usage laws and replay

The final slide opens the local HTML demo and its authored offline kernels.

FINAL SLIDE · LOOPBACK ONLY

http://127.0.0.1:8765/

FROM REPOSITORY ROOT

```
python3 demo/server.py
```

01 Hold public inputs fixed

02 Compare aggregate and ordered usage

03 Open the hidden-state trace-law story

Offline authored kernels · no provider calls · URL works while the local server is running

SOURCE demo/README.md